Relatório de Preparação

Perguntas e Respostas — Entrevista Técnica

Avanade — Desenvolvedor(a) Back-End Júnior (R00348516)

Candidato: Lucas Williams César Pires

Entrevista: quarta-feira 26/08 às 13:00 (com desenvolvedor)

Estude em voz alta. As respostas são modelos — adapte com exemplos reais da MV e dos seus repositórios GitHub.

Sumário

- 1. Como usar este material
- 2. Pitch de 60 segundos
- 3. Java, Spring Boot e APIs REST
- 4. JPA, Hibernate e banco de dados
- 5. Arquitetura (camadas, hexagonal, microserviços)
- 6. SQL Server, Redis e performance
- 7. Testes, TDD e qualidade
- 8. CI/CD, Git e DevOps
- 9. Azure, OpenShift e cloud (glossário + respostas)
- 10. Experiência na MV e projetos GitHub
- 11. Comportamentais (STAR)
- 12. Como falar das lacunas
- 13. Perguntas para você fazer ao entrevistador
- 14. Checklist do dia da entrevista

1. Como usar este material

Leia cada pergunta, tape a resposta e responda em voz alta. Depois compare. Não decore palavra por palavra — memorize a estrutura e use exemplos seus.

- Prioridade alta: seções 3, 4, 5, 10 e 11 (o que o dev mais cobra).
- Prioridade média: seções 6, 7 e 8.
- Prioridade glossário: seção 9 (honestidade > inventar experiência).

Regra de ouro: se não souber, diga o que sabe, o que faria e como validaria. Nunca invente.

2. Pitch de 60 segundos

Pergunta típica: “Me conta um pouco sobre você / sua experiência.”

Sou desenvolvedor Java júnior na MV, atuando com Spring Boot, APIs REST, Oracle e Angular. No dia a dia investigo bugs em produção, integro front, back e banco, e trabalho com Git, Jira, Azure DevOps e Docker. Tenho projetos públicos no GitHub com JPA, Security, validação e testes. Estou aprofundando SQL Server, Redis, arquitetura hexagonal, microserviços e CI/CD — stack alinhada à vaga — e busco evoluir em um ambiente Microsoft/Azure como o da Avanade.

Ensaie até caber em ~60 segundos, sem correr.

3. Java, Spring Boot e APIs REST

P1. Explique o fluxo de uma requisição no Spring Boot.

Resposta modelo:
A requisição HTTP chega no DispatcherServlet, que roteia para o Controller anotado (@RestController + @GetMapping/@PostMapping). O Controller recebe PathVariable/RequestBody, chama o Service (regra de negócio) e o Service usa o Repository (Spring Data JPA) para acessar o banco. O retorno vira JSON via Jackson e sai com o status HTTP adequado (200, 201, 404...).

Dica: Cite um endpoint seu, ex.: GET /users/{id} do api_rest.

P2. Qual a diferença entre @Controller e @RestController?

Resposta modelo:
@Controller é para MVC com views. @RestController = @Controller + @ResponseBody: o retorno do método já é o corpo da resposta HTTP (geralmente JSON), típico de API REST.

P3. O que são status HTTP e quando usar cada um?

Resposta modelo:
200 OK (sucesso GET/PUT), 201 Created (POST criou recurso), 204 No Content (DELETE ok sem corpo), 400 Bad Request (validação/entrada inválida), 401 Unauthorized (não autenticado), 403 Forbidden (sem permissão), 404 Not Found, 409 Conflict (regra de negócio), 500 Internal Server Error (erro inesperado no servidor).

P4. O que é DTO e por que usar?

Resposta modelo:
DTO (Data Transfer Object) é um objeto só para transporte de dados na API. Evita expor a entidade JPA diretamente (segurança, acoplamento, lazy loading estourando JSON) e permite formatar exatamente o contrato do endpoint.

Dica: Se no seu projeto ainda devolve entidade, diga que hoje usaria DTO + mapper.

P5. Como você trata exceções numa API Spring?

Resposta modelo:
Com @ControllerAdvice + @ExceptionHandler centralizo o tratamento: ResourceNotFound → 404, MethodArgumentNotValidException → 400 com detalhes de validação, e um handler genérico → 500 sem vazar stack trace sensível para o cliente.

P6. O que é injeção de dependência e @Autowired?

Resposta modelo:
O Spring cria e injeta as dependências (beans) em vez de você dar new. Isso facilita teste (mocks) e desacoplamento. Prefiro injeção por construtor (immutável e obrigatória) em vez de @Autowired em campo.

P7. Diferença entre PUT e PATCH.

Resposta modelo:

PUT atualiza o recurso de forma completa (substitui). PATCH atualiza parcialmente. POST cria; GET lê; DELETE remove. APIs devem ser semânticas e, no GET, idempotentes.

P8. O que você sabe de Spring Security / JWT?

Resposta modelo:

Já pratiquei em projetos com autenticação JWT e OAuth2: o cliente envia token no header Authorization, um filtro valida o token e o contexto de segurança define o usuário e as roles. Bean Validation (@Valid, @NotBlank) complementa na entrada da API.

Dica: Cite validacao_e_seguranca ou dscommerce se perguntarem evidência.

4. JPA, Hibernate e banco de dados

P9. O que é JPA / Hibernate?

Resposta modelo:
JPA é a especificação de ORM em Java; Hibernate é a implementação mais comum. Mapeio classes (@Entity) para tabelas e uso Spring Data JPA (interfaces Repository) para CRUD sem escrever SQL repetitivo — ainda posso usar JPQL ou native query quando preciso.

P10. Explique @Entity, @Id, @GeneratedValue, relacionamentos.

Resposta modelo:
@Entity marca a classe persistente; @Id a chave; @GeneratedValue(IDENTITY) deixa o banco gerar o id. @ManyToOne / @OneToMany / @ManyToMany modelam relacionamentos. No api_rest, User tem @ManyToOne com Department via department_id.

P11. Lazy vs Eager — qual a diferença?

Resposta modelo:
LAZY carrega o relacionamento só quando acessado; EAGER carrega junto com a entidade. O padrão em @ManyToOne costuma ser EAGER (no JPA clássico) e em coleções LAZY. Lazy mal usado fora de transação causa LazyInitializationException; Eager demais causa over-fetch.

P12. O que é o problema N+1 e como evitar?

Resposta modelo:
N+1: 1 query busca a lista e depois N queries buscam o relacionamento de cada item. Evito com JOIN FETCH na JPQL, @EntityGraph, ou DTO/projeção. Também reviso logs com spring.jpa.show-sql para detectar.

P13. Para que serve @Transactional?

Resposta modelo:
Define o limite de uma transação: tudo commit ou rollback. Uso no Service para operações que alteram mais de uma tabela ou precisam de consistência. Leitura pode usar readOnly=true. Cuidado com self-invocation e exceções checked.

P14. Por que findById().get() é perigoso?

Resposta modelo:
Optional.get() sem checagem estoura NoSuchElementException se o id não existir. O correto é orElseThrow(() -> new ResourceNotFoundException(...)) e devolver 404.

Dica: Isso mostra maturidade — mencione melhoria no seu código atual.

P15. Diferença entre SQL e JPQL.

Resposta modelo:
SQL fala com tabelas/colunas do banco. JPQL fala com entidades/atributos Java. JPQL é mais portável entre bancos; native query uso quando preciso de dialeto específico ou performance.

P16. O que é ddl-auto (update, validate, none)?

Resposta modelo:

Controla se o Hibernate altera o schema. update é ok em lab/dev; em produção prefiro validate/none + migrations (Flyway/Liquibase) para mudanças versionadas e seguras.

5. Arquitetura

P17. Repository vs Service — responsabilidades?

Resposta modelo:
Repository: acesso a dados (CRUD, queries). Service: regra de negócio, orquestração, transações. Controller: HTTP — recebe request, chama service, devolve response. Não coloco regra de negócio no Controller nem SQL no Controller.

P18. O que é arquitetura hexagonal (ports & adapters)?

Resposta modelo:
O domínio/caso de uso fica no centro. Ports são contratos (interfaces). Adapters são implementações: HTTP (entrada), JPA/Redis (saída). Benefício: testar regra sem subir Spring/banco, e trocar tecnologia sem reescrever o domínio.

Desenho rápido: HTTP Adapter → Port In → Domínio → Port Out → JPA/Redis Adapters

P19. Monólito vs microserviço — quando usar cada um?

Resposta modelo:
Monólito: um deploy, mais simples no início, bom para time pequeno e domínio ainda nebuloso. Microserviço: serviços independentes por bounded context — escala e deploy isolados, mas traz rede, consistência eventual, observabilidade e complexidade operacional. Para júnior: começo modular no monólito e extraio serviço quando o contexto estiver claro.

P20. O que é API RESTful de verdade para você?

Resposta modelo:
Recursos nomeados por substantivos (/users), verbos HTTP corretos, status adequados, payload previsível (DTO), versionamento se necessário, documentação (OpenAPI) e tratamento de erro consistente. Não é só “ter JSON num @RestController”.

P21. Como trataria falha ao chamar um serviço externo?

Resposta modelo:
Timeout configurado, tratamento de erro (4xx/5xx), log com correlação, retry com cuidado (só em erros transitórios), fallback quando fizer sentido, e não engolir exceção em silêncio. Em cenário avançado: circuit breaker (Resilience4j).

6. SQL Server, Redis e performance

P22. Você tem experiência com SQL Server?

Resposta modelo:

Minha base forte é Oracle e PostgreSQL na MV e nos projetos. SQL Server é relacional — a modelagem, joins, índices e JPA continuam iguais; mudam dialeto, tipos e o driver JDBC. Já estudei a configuração de datasource SQL Server no Spring e consigo transferir rápido.

P23. O que é Redis e como usaria numa API?

Resposta modelo:

Redis é um banco em memória, ótimo para cache. Padrão cache-aside: no GET, olho a key no Redis; se miss, busco no SQL Server/Oracle, gravo no Redis com TTL e retorno; no update, invalido a key. Reduz carga no banco em leituras quentes (ex.: catálogo de produto).

P24. Quando NÃO usar cache?

Resposta modelo:

Dado muito volátil, dado sensível sem política clara, ou quando a complexidade de invalidação é maior que o ganho. Cache errado é bug difícil (dado stale).

P25. Como você analisa performance num endpoint lento?

Resposta modelo:

1) Reproduzir e medir tempo. 2) Ver SQL gerado (N+1, full scan). 3) Checar índices. 4) Avaliar se cabe cache. 5) Paginação em listas grandes. Na MV já investigo bugs e lentidão olhando integração e banco — o método é o mesmo.

P26. O que é índice e por que importa?

Resposta modelo:

Estrutura que acelera busca por colunas de filtro/join. Sem índice certo, o banco faz scan. Índice demais atrasa escrita. Em entrevista júnior, mostrar que você pensa nisso já diferencia.

7. Testes, TDD e qualidade

P27. O que é TDD?

Resposta modelo:
Test-Driven Development: Red → Green → Refactor. Escrevo o teste que falha, implemento o mínimo para passar, depois limpo o código. Ajuda a desenhar a API do Service/domínio e evita regressão.

P28. Diferença entre teste unitário e de integração.

Resposta modelo:
Unitário: isola uma classe (mocks com Mockito), rápido, testa regra. Integração: sobe contexto Spring / banco (ou Testcontainers) e valida o fio Controller–JPA. Os dois se complementam; unitário em volume, integração nos fluxos críticos.

P29. O que você testaria num Service de criar usuário?

Resposta modelo:
Caminho feliz (salva e retorna), e-mail duplicado (regra), validação de entrada, e se o repository foi chamado corretamente. Com JaCoCo acompanho cobertura, mas priorizo comportamento importante, não % cega.

Dica: Cite dscommerce_jacoco se pedirem evidência.

P30. O que é mock?

Resposta modelo:
Objeto falso que simula dependência. No unitário do Service, mocko o Repository para não precisar de banco e controlar o retorno (quando id existe, quando não existe…).

8. CI/CD, Git e DevOps

P31. O que é CI e o que é CD?

Resposta modelo:
CI (Continuous Integration): a cada push/PR, build + testes automáticos para pegar quebra cedo. CD (Continuous Delivery/Deployment): caminho automatizado até homolog/produção com mais controles. Na MV uso Azure DevOps; nos labs estudo GitHub Actions (checkout → JDK → mvn test).

P32. Como montaria um pipeline CI simples para Java?

Resposta modelo:
Trigger no push/PR → checkout → setup Java 17 → cache Maven → mvn -B test (e opcionalmente package). Se falhar teste, o merge não segue. Depois posso acrescentar build de imagem Docker e deploy.

P33. Como é seu fluxo com Git?

Resposta modelo:
Branch por feature, commits claros, PR/MR com review, merge na main. Uso GitLab/Azure DevOps no trabalho e GitHub nos projetos pessoais. Evito commit grande sem contexto; prefiro histórico legível.

P34. Para que serve Docker no seu dia a dia?

Resposta modelo:
Padronizar ambiente: subir app/dependências (banco, Redis) sem “na minha máquina funciona”. Docker Compose orquestra vários containers juntos no lab local.

9. Azure, OpenShift e cloud

Avanade é parceira Microsoft — vão valorizar interesse e vocabulário correto, não experiência inventada. Seja honesto.

P35. O que você sabe de Azure?

Resposta modelo:
Tenho Azure DevOps na prática (pipelines/boards no trabalho). Estudei os serviços que fazem sentido para API Java: App Service ou Container Apps (hospedar), Azure SQL (SQL Server gerenciado), Azure Cache for Redis, Key Vault (segredos) e Application Insights (monitoramento). Ainda não tenho deploy produtivo meu na Azure, mas entendo o papel de cada um.

P36. O que é OpenShift?

Resposta modelo:
Plataforma Kubernetes enterprise da Red Hat: orquestra containers (Pod, Deployment, Service) e expõe aplicação (Route). Conceito base é o mesmo do Kubernetes, com ferramentas e segurança corporativas por cima.

Glossário rápido (decore)

- App Service / Container Apps — hospeda API web ou containers.
- Azure SQL — SQL Server gerenciado na nuvem.
- Azure Cache for Redis — Redis gerenciado.
- Key Vault — senhas e connection strings.
- Application Insights — telemetria e diagnóstico.
- OpenShift — Kubernetes enterprise (Pod, Deployment, Service, Route).

10. Experiência na MV e projetos

P37. O que você faz na MV no dia a dia?

Resposta modelo:

Desenvolvimento Java/Spring: APIs REST, manutenção, correção de bugs em produção, integração com Angular no front e Oracle no banco (SQL/PLSQL). Trabalho com Git/GitLab, Jira, Azure DevOps e Docker, sempre buscando entender o fluxo ponta a ponta antes de mudar código em produção.

Dica: Prepare 1 exemplo concreto de bug ou feature (sem dados sensíveis).

P38. Me explica um projeto do seu GitHub.

Resposta modelo:

Exemplo api_rest: API Spring Boot 3 / Java 17 com JPA, PostgreSQL, entidades User e Department (@ManyToOne), endpoints GET lista, GET por id e POST. Hoje eu evoluiria com camada Service, DTO, Exception Handler e testes — e é exatamente esse tipo de evolução que me interessa na Avanade.

P39. Por que quer sair / por que a Avanade?

Resposta modelo:

Quero crescer em backend com boas práticas, arquitetura, cloud Microsoft e times maduros de engenharia. A vaga pede Spring, SQL Server, Redis, hexagonal e CI/CD — é o caminho que já estou estudando. Avanade combina minha base Java com o ecossistema Azure.

P40. Qual sua maior fraqueza técnica?

Resposta modelo:

Ainda estou aprofundando microserviços, Redis em produção e cloud Azure além do DevOps. Compenso com estudo dirigido, labs no GitHub e honestidade sobre o que já pratiquei vs o que estudei. Na MV minha força é entregar e depurar no stack Java/Spring/Oracle.

P41. Onde se vê em 2 anos?

Resposta modelo:

Como backend sólido: APIs bem desenhadas, testes, CI, noção forte de dados e cache, contribuindo em code review e evoluindo para arquitetura (hexagonal/microsserviços) com mais autonomia.

11. Comportamentais (STAR)

Método STAR: Situação → Tarefa → Ação → Resultado. Preencha com casos REAIS seus antes da entrevista.

P42. Conte um bug difícil que você resolveu.

Estrutura: situação em produção/homolog → o que era sua responsabilidade → como investigou (log, SQL, integração front/back) → o que corrigiu e o impacto.

Resposta-guia: “Tivemos [sintoma]. Eu precisei [tarefa]. Investiguei [passos]. A causa era [raiz]. Corrigi [ação] e o resultado foi [usuário/sistema estável / tempo].”

P43. Como você aprende uma tecnologia nova sob prazo?

Defino o mínimo necessário para a tarefa, estudo doc oficial + 1 exemplo, faço um spike pequeno, valido com teste ou prova manual, e só então integro na feature. Documento o essencial para o time. Evito tutorial infinito sem entregar.

P44. Situação em que pediu ajuda / colaborou.

Mostre humildade + ownership: “bloqueei X tempo, juntei evidências (erro, o que já tentei), pedi ajuda ao colega/sênior com pergunta objetiva, implementei a solução e compartilhei o aprendizado.”

P45. Como prioriza correção vs feature?

Impacto no usuário e risco em produção primeiro. Bug crítico > feature. Alinho com o time/PO no Jira. Se feature e bug competem, explico trade-off com dados.

P46. Conte um conflito / feedback difícil.

Foque em profissionalismo: ouviu, esclareceu expectativa, ajustou comunicação ou código, e manteve o objetivo do time. Evite falar mal de colega ou empresa.

12. Como falar das lacunas

SQL Server

“Venho de Oracle/PostgreSQL; a lógica SQL é a mesma. Já estudei o driver e o datasource do SQL Server no Spring e consigo produzir rápido.”

Redis

“Usaria cache-aside no GET com TTL e invalidação no update. Ainda não operei Redis em produção na MV, mas o padrão está claro e pratiquei o conceito no estudo.”

Hexagonal

“Domínio no centro, ports como contratos, adapters para HTTP e banco — facilita teste isolado. Hoje trabalho em camadas; sei como migrar um caso de uso para esse estilo.”

Azure / OpenShift

“Tenho Docker e Azure DevOps na prática. Estudei App Service, Azure SQL, Redis Cache, Key Vault, Insights e OpenShift como Kubernetes enterprise. Não vou inventar deploy que não fiz.”

Inglês

Se perguntarem: seja honesto sobre o nível e diga que lê documentação técnica e está evoluindo comunicação. Avanade valoriza times globais — mostre disposição.

13. Perguntas para o entrevistador

Faça 1 ou 2 no final — demonstra interesse real:

- Quais são os principais desafios técnicos do time hoje?
- Como é o fluxo de code review e CI/CD no projeto?
- No dia a dia do júnior, o quanto é API nova vs manutenção/bug?
- Quais tecnologias o time mais usa no backend atualmente (SQL Server, Redis, mensageria)?
- Como é o onboarding técnico nas primeiras semanas?

14. Checklist do dia da entrevista

- Entrar 10 minutos antes (link, câmera, microfone, fone testados).
- Repo favorito aberto no navegador.
- Este PDF / anotações de 1 página ao lado (não ler na tela).
- Pitch ensaiado 1–2 vezes na manhã — sem estudar assunto novo.
- Água, ambiente silencioso, celular no silencioso.
- Se travar: o que sei → o que faria → como validaria.
- Terminar com uma pergunta boa.
- Agradecer o tempo do entrevistador.

Você já tem base de júnior em produção. O diferencial é clareza.

Boa entrevista, Lucas.